ACSI-BD

‘70s : **Edgar Frank** créer le modèle relationnel.

Définition : Une base de données (*BD*) est une structure de données qui permet de gérer des données persistantes.

Elle a des **primitives d’accès** (permet d’accéder aux données en question), ainsi que des **constructeurs** (permet de créer les données), et des **destructeurs** (supprime les données).

Un système de **g**estion de **b**ase de **d**onnées (*SG-BD* en français) est un logiciel permettant de créer et manipuler de telles données.

Un langage d’interrogation : « ***S**tructural **Q**uery **L**anguage* » (AKA **SQL**).

Une *BD* c’est un ‘ensemble de table’ (penser Excel), exemple :

attribut

ÉTUDIANTS

tuple

numINE	nomEtd	prénomEtd	dptEtd	grpEtd
2382996	Chabellard	Félix	INFO	S1D
1954664	Dela	Michèle	GE2I	S2A
7892240	Froy	Rémy	HSE	S4F
...	...	...	...	...

Chaque colonnes est un ***attribut*** de la table et possède un domaine (en type).

Ex : *numINE* est un **entier** | *nomEtd* est une **chaîne de caractères**

Chaque ligne est un ***tuple*** et regroupe les informations d’un étudiant.

Ex : *(2382996, Chabellard, Félix, INFO, S1D)*

Dans chaque table, on a besoin d’une information qui permet de distinguer les lignes des autres.

Ex : **Ici** c’est *numINE*

/!\ ce n’est pas le nom et/ou prénom de l’étudiant qui peut jouer le rôle /!
→ les homonymes (= même numéro entre étudiants)
→ c’est trop long (un numéro est bien plus court)

On appelle cette information la **clé primaire** de la table, on décrit la structure de notre table comme ceci :

ÉTUDIANTS(*numINE, nomEtd, prénomEtd, dptEtd, grpEtd***)**

C'est le schéma de la table Étudiants.

On donne **le schéma relationnel** d'une *BD* en donnant la liste des schémas de ses tables.

Bases Mathématiques : Le modèle relationnel repose sur des bases mathématiques fortes (*cf. Moodle*).

- Une table est une relation entre plusieurs domaines ;
- Chaque ligne est un élément du produit cartésien des domaines ;

Contraintes d'intégrité : Assure la cohérence de la *BD*.

Ex : tout étudiant doit avoir un nom
→ *Attribut obligatoire (champ * sur les formulaires)*
→ *Étudiant numEtd ne peut être 'NULL'*

Contraintes de référence : On assigne un code à une ville au lieu du nom par exemple, afin de ne pas avoir à le changer pour chaque étudiant dans l'éventuel cas ou la ville changerait de nom (*codeVille* au lieu de *nomVille*).

SQL - Requêtes Simples : structure générale

SELECT: < *liste_des_attributs_qui_nous_intéressent* >

FROM: < *la_table_concernée* >

[WHERE: < *condition_de_sélection_des_lignes* >

[ORDER BY: < *critère_de_tri* >

|Notre table : **ETU(***code*Etd, *nom*Etd, *prénom*Etd, *dateNé*Etd, *annéeBac*Etd, *sérieBac*Etd**)**

|Projection : (*« oublier »* des colonnes)

SELECT, nomEtd, prénomEtd, FROM, ETU...

Ex : SELECT *
FROM ETU
ORDER BY nomEtd ASC

→ on prends toutes les colonnes de la table étudiantes et on tri les lignes par « nomEtd » dans l'ordre croissant (ASC)

Requêtes simples avec sélection : (rien avoir avec SELECT) Ici, l'objectif est de ne garder que les lignes de la table qui nous intéressent.

→ On utilise une condition pour cela avec le mot clé WHERE.

Ex : *les étudiants ayant eu leurs BAC en 2020*
SELECT +
FROM ETU
WHERE annéeBacEtd = 2020

On a aussi des opérateurs logiques (***AND***, ***OR***, ***NOT*** = *ET*, *OU*, *NON*).

Requêtes simples avec projection & sélection :

Ex : *les noms + date de naissance des étudiants prénommés « Wati Bg » et ayant un BAC ES, triés par date de naissance du plus jeune au plus âgé.*
SELECT nomEtd, dateNéEtd
FROM ETU
WHERE prénomEtd = 'Wati Bg' AND mêmeBacEtd = 'ES'
ORDER BY dateNéEtd DES décroissant

|Remarque :

- L'ordre des mots-clés (*SELECT, FROM, WHERE, ORDER BY*) est important ;
- Les chaînes de caractères sont entre apostrophes (ex: *'ES', 'Wati Bg'*, …) ;
- On peut utiliser les parenthèses pour les conditions (*a = 5 AND b = 12*) .

Les mots-clés distinct : supprime les doublons.

Ex : SELECT < *attributs* > → résultats avec doublons
SELECT DISTINCT < *attributs* > → résultats sans doublons

|Remarque : la liste est triée pour des raisons de simplicité.

Fonctions d'agrégations : On calcule des « choses » sur une table, le résultat est un nombre = une table à 1 ligne et 1 colonne. La colonne (attribut) n'a pas de nom si on ne lui en donne pas.

On lui donne un nom avec le mot-clé *AS*.

La Fonction COUNT :

- Nombre de lignes de la table musicien.

```
SELECT  COUNT( * )  AS  'Nb Musiciens'
FROM  Musiciens
```

Nb Musiciens
10560

- Nombre de lignes de **Musiciens** ayant un NomMusicien non *NULL*.

```
SELECT  COUNT( NomMusicien )  AS  'Nb de Noms'
FROM  Musiciens
```

- Cette requête est équivalente à la 1ère.

```
SELECT  COUNT( Code_Musicien )  AS  'Nb de dés'
FROM Musiciens
```

En effet, *Code_Musicien* est une clé et donc est non *NULL*.

✗ Nombre de prénoms distinct de Musiciens.

SELECT COUNT(DISTINCT Prénom_Musicien) […]
[…] AS ‘Nb prénoms distincts’
FROM Musiciens

Autres Fonctions : voir ci-dessous

SUM ajoute des nombres
AVG calcule la moyenne
MIN donne le minimum
MAX donne le maximum

|Remarques :

- Vous pouvez toujours utiliser WHERE si vous voulez ;

*SELECT COUNT(*) AS ‘Nb Musiciens nés en 1920’*
FROM Musiciens
WHERE Année_Naissance = 1920

- On peut aussi faire des calculs ;

SELECT MAX(Prix) - MIN(Prix) AS ‘écart max’
FROM Produit

/!\ Mettre des espaces autour du signe « - » /!\

SELECT MAX(Poids/1000) AS ‘Poids Max en tonnes’
FROM Produits

Note : On a pas besoin de faire des agrégations pour faire des calculs

SELECT CodeVol, HeureArrivée – HeureDépart AS ‘durée’
FROM Vols

/!\ Ceci ne marche pas : /!\

SELECT Titre, MAX(Prix) AS '...'

FROM Albums

car « Titre » est une colonne et plein de lignes

Fin?